

ASE: 一个Python脚本, 打通DFT/分子动力学/机器学习

原创 嘻歪歪 智算催化 2025年11月28日 08:01 浙江



点击卡片 关注我们



智算催化

高质量最新AI for Chemistry论文解读。聚焦AI/机器学习/大模型与催化材料计算设计的...
139篇原创内容

公众号



目录 (一级标题)

- 一、ASE 是什么? 谁开发的? 能干什么?
- 二、不同系统下的 ASE 安装方法
- 三、ASE 的常用功能与命令行技巧
- 四、ASE × 常见理论计算: DFT、GROMACS 与 workflow
- 五、ASE × 机器学习 / 深度学习: 几条可落地路径
- 六、小结: 把 ASE 当“中枢”, 把工具链串成一个系统

一、ASE 是什么? 谁开发的? 能干什么? 🔍

1.1 ASE 的“身份”: 原子模拟的 Python 操作系统

ASE 全称 **Atomic Simulation Environment**, 是一个用 Python 编写的开源原子模拟环境库。最早由丹麦技术大学 (DTU) 物理系 CAMD 小组发起, 现在由国际社区持续维护。

它干的事情可以总结为三句话:

- 用一个统一的 `Atoms` 对象描述“原子体系”;
- 用一个统一的 `Calculator` 接口对接各种 DFT / 力场 / ML 势;
- 用 Python 脚本统一管理结构搭建、任务提交、结果分析、画图与可视化。

常见能力包括:

- 建分子、晶体、表面、缺陷、纳米簇;
- 调用 VASP / GPAW / Quantum ESPRESSO / GROMACS 等做能量、力、应力计算;
- 做几何优化、分子动力学 (MD)、NEB 反应路径、弹性常数等;
- 用 `ase gui` 和命令行工具浏览结构与轨迹、做格式转换。

1.2 和其它工具怎么配合?

- 和 **pymatgen**:
 - ASE 更偏“建模 + 调引擎 + workflow”;
 - pymatgen 更偏“结构分析 + 材料数据库 + 相图 / Pourbaix 图”等后处理;
 - 实战中经常是: **ASE 负责算, pymatgen 负责分析**。
- 和 **MDAnalysis / OVITO**:
 - ASE 可以做简单 MD 与轨迹操作;
 - 大型蛋白 / 聚合物长时间 MD 的统计分析, MDAnalysis 会更专业; OVITO 更强在可视化。
- 和商业平台 (**QuantumATK 等**) :
 - 商业平台集成 GUI+计算, 适合快速入门, 但封闭、不好改;
 - ASE 完全开源, 很适合做自动化高通量、对接机器学习、自定义 workflow。

1.3 支持哪些系统?

- **Linux**: 服务器、集群、超算上的主力平台;
- **macOS**: 本地写脚本 + GUI 调试体验不错;
- **Windows**: 通过 Anaconda / Miniforge / WSL 就能无压力使用。

二、不同系统下的 ASE 安装方法

2.1 Windows + Anaconda: 最稳妥路线

1. 安装 Anaconda 或 Miniconda / Miniforge;
2. 打开 Anaconda Prompt:

```
conda create -n ase_env python=3.11 -y
conda activate ase_env
conda install -c conda-forge ase
# 或者: pip install --upgrade ase
```

3. 检查:

```
python -c "import ase; print(ase.__version__)"
ase info -h
```

2.2 WSL (Windows Subsystem for Linux)

1. 在 WSL(Ubuntu 等) 中安装 Miniconda / Miniforge;

2. 和 Linux 完全一致:

```
conda create -n ase_env python=3.11 -y
conda activate ase_env
conda install -c conda-forge ase
```

3. 如需 GUI, 可在 Windows 装 VcXsrv / X410 / MobaXterm, 自行配置 `DISPLAY` (后文会讲)。

2.3 Linux 服务器: 普通用户的 Conda 安装 (最推荐)

无需 root, 非常适合实验室用户:

```
# 1. 创建独立环境
conda create -n ase_env python=3.11 -y
conda activate ase_env

# 2. 安装 ASE
conda install -c conda-forge ase

# 3. 如需 GUI + 画图支持
conda install -c conda-forge tk matplotlib pillow pyqt pyglet
```

2.4 Linux 服务器: 管理员统一安装 + module 管理

这种适合机房统一环境:

```
# 管理员侧 (示例)
sudo apt install python3-ase # 或 sudo pip install ase
```

然后写一个 `modulefile` 暴露给用户:

```
##Module1.0

set root /usr/lib/python3/dist-packages

prepend-path PYTHONPATH $root

setenv ASE_HOME $root
```

用户使用时:

```
module load ase/3.x

python -c "import ase; print(ase.__version__)"
```

2.5 macOS: 本地开发与调试首选

```
conda create -n ase_env python=3.11 -y

conda activate ase_env

conda install -c conda-forge ase
```

然后直接:

```
ase gui
```

三、ASE 的常用功能与命令行技巧

3.1 Python 侧核心模块: Atoms + IO + Calculators

几个最常用模块:

```
from ase import Atoms

from ase.build import molecule, bulk, fcc111

from ase.io import read, write

# 分子

h2o = molecule("H2O")

# 体相 Si

si = bulk("Si", "diamond", a=5.43)

# Pt(111) 表面, 2x2, 4 层, 上下真空 10 Å

slab = fcc111("Pt", size=(2, 2, 4), vacuum=10.0)

# 读 / 写文件: 支持 POSCAR / CIF / xyz / traj 等

atoms = read("POSCAR")

write("out.cif", atoms)

write("movie.traj", [slab, slab]) # 写成 ASE 轨迹
```

3.2 ase 命令行工具:

命令行统一入口是：

```
ase <子命令> [参数...]
```

常见子命令 & 示例：

1. 查看结构信息：

```
ase info POSCAR
```

2. 格式转换：

```
ase convert in.cif out.vasp ..... # CIF → POSCAR
ase convert md.traj md.xyz ..... # ASE 轨迹 → xyz
ase convert conf.gro conf.pdb ..... # GROMACS gro → PDB
```

3. 快速构建结构：

```
# 构建一个 CO2 分子，输出 xyz
ase build CO2 co2.xyz

# 构建 2×2×4 的 Pt(111) 表面（有的版本写法略有不同）：
ase build Pt111 2 2 4 10.0 pt111.vasp
```

4. NEB 分析示例：

```
# 假设 neb.traj 是 NEB 跑完的轨迹
ase nebplot neb.traj
```

5. 维度判断 (0D/1D/2D/3D)：

```
ase dimensionality structure.cif
```

6. 数据库操作 (ASE db)：

```
ase db mycalc.db -l ..... # 列出条目
ase db mycalc.db energy<0 ..... # 查询能量小于 0 的结构
```

广告

研才

非全日制硕士扩招

2年制 | 学费一万元

2年制（无需到校）+学费7980元+录取线低+证书齐全（学信网可查）

 研才教育

[查看详情](#)

这些命令在写 workflow 时非常方便，尤其是[批处理格式转换](#)、[快速自查文件内容](#)。

3.3 GUI 可视化 & X11 / DISPLAY 设置要点

3.3.1 本地 (Windows / macOS)

本机安装好 ASE 后：

```
ase gui POSCAR
```

一般就会弹出窗口。

3.3.2 Linux 服务器 + X11 转发

在服务器上跑 `ase gui`，需要把图形“转发”回本地显示器，靠的是 X11。

客户端：

- Linux / mac：自带或安装 XQuartz；
- Windows：推荐 MobaXterm（自带 X server）、Xming、VcXsrv 等。

连接方式：

```
# 本地终端 / MobaXterm
ssh -Y username@server    # 或 ssh -X

# 登录后在服务器上
echo $DISPLAY

# 正常情况下会输出类似：localhost:10.0

ase gui slab.traj
```

⚠ DISPLAY 的常见坑：

有些人会在 `~/.bashrc` 里写：

```
export DISPLAY=":0"
```

这会把远程 SSH 自动设置好的 DISPLAY 覆盖掉，导致 `TclError: no display name and no $DISPLAY environment variable`。

更稳妥的写法是 **只在 DISPLAY 为空的时候设置**（一般在本地物理机上）：

```
[ -z "$DISPLAY" ] && export DISPLAY=:0
```

如果你是从 Windows / mac SSH 到服务器，并开启了 X11 转发，**通常不需要手动设置 DISPLAY，让 SSH 自动帮你设就好。**

3.3.3 Windows + MobaXterm 示例

1. 在 MobaXterm 新建 SSH Session：勾选 “X11 forwarding”；
2. 登录服务器后：

```
echo $DISPLAY ..... # 确认非空
ase gui interface.gro # 例如查看 GROMACS 的界面结构
```

四、ASE × 常见理论计算：DFT、GROMACS 与 workflow

这一节重点讲：**ASE 如何对接常见的量化 / 力场软件**，包括 **DFT（以 VASP 为例）** 和 **GROMACS**，并给出脚本设置与注释。

4.1 ASE 与 DFT：总体思路

对于 DFT 代码（VASP / GPAW / Quantum ESPRESSO 等），ASE 一般做几件事：

1. 用 `Atoms` 构建或读入结构；
2. 创建对应的 `Calculator`（例如 `Vasp` / `GPAW` / `Espresso` 等）；
3. 把 `calculator` 挂在 `atoms.calc` 上；
4. 调用 `atoms.get_potential_energy()`、`get_forces()`、优化器等进行计算；
5. ASE 负责生成输入、调用外部程序、解析输出。

下面就用 **VASP 几何优化** 做一个比较完整的示例。

4.2 示例一：用 ASE + VASP 做几何优化（含详细注释）

假设当前目录有一个 VASP 的 `POSCAR`，我们想用 ASE 控制 VASP 做结构优化。

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

from ase.io import read, write
from ase.calculators.vasp import Vasp
from ase.optimize import BFGS

# 1. 读入初始结构
atoms = read("POSCAR") # 也可以是 CIF / XYZ 等

# 2. 设置 VASP 计算器
calc = Vasp(
    # --- 基本收敛与精度 ---
    encut=520,          # 平面波截断能 (eV)，一般比 POTCAR 推荐值略高
    xc="PBE",           # 交换-相关泛函
    kpts=[4, 4, 1],     # k 点网格，这里是 4×4×1
    ediff=1e-5,         # 电子步收敛标准
    ediffg=-0.03,       # 力收敛标准 (eV/Å)，负数表示力小于 |ediffg| 即收敛

    # --- 自洽场相关 ---
    ismear=0,           # 高斯展宽方法，一般绝缘体用 0
    sigma=0.05,        # 展宽宽度
    istsart=0,          # 从头开始计算
    icharg=2,           # 从原子超位势生成初始电荷密度

    # --- 计算类型与离子优化 ---
    isif=3,             # 3: 优化原子 + 晶格; 2: 只优化原子
    nsw=100,            # 最大离子步数
    ibrion=2,           # 2: 共轭梯度法; 1: 准牛顿; -1: 无离子步
    potim=0.5,          # 离子步长 (时间步)

    # --- 其他选项 ---
    lwave=False,        # 不写 WAVECAR (节省空间)
    lcharg=False,       # 不写 CHGCAR
    lreal="Auto",       # 实空间投影加速
)

# 3. 把计算器挂到原子体系上
atoms.calc = calc

# 4. 使用 ASE 自带优化器进行几何优化
```

```

dyn = BFGS(atoms, trajectory="opt.traj", logfile="opt.log")
dyn.run(fmax=0.03)      # 最大力小于 0.03 eV/Å 认为收敛

# 5. 输出最终结构
write("CONTCAR_ase", atoms)
print("Final energy:", atoms.get_potential_energy(), "eV")

```

这个脚本的优点是：

- VASP 的所有重要 INCAR 参数都用 Python 变量可控；
- 同一套逻辑可以轻松换成 GPAW / Espresso 等 calculator；
- 可以和后续的 MD / NEB / ML 工作流自然串联。

4.3 示例二：用 ASE 做一个简单的 DFT MD

在上面几何优化的基础上，如果想做一个简单的 **NVE or NVT DFT MD**，可以用 ASE 的 MD 模块。下面以“理想示例”为主（实际 DFT MD 会比较贵）：

```

from ase.io import read

from ase.calculators.vasp import Vasp
from ase.md.verlet import VelocityVerlet
from ase import units

atoms = read("CONTCAR_ase") # 用刚刚优化好的结构

# 重新设置一个较简化的 DFT 计算器（MD 一般不需要超高收敛）
atoms.calc = Vasp(
    encut=400,
    xc="PBE",
    kpts=[2, 2, 1],
    ismear=0,
    sigma=0.1,
    isif=2,
    nsw=0,      # 离子步由 ASE 管，不再由 VASP 接管
    ibrion=-1,  # -1: 不做离子优化，适合 MD
    lwave=False,
    lcharg=False,
)

# 设置一个简单的 NVE MD（演示用）
timestep = 1.0 * units.fs # 时间步 1 fs

dyn = VelocityVerlet(atoms, dt=timestep, trajectory="md.traj", logfile="md.log")

```

```
for step in range(100):    # 100 步示例
    dyn.run(1)
    E = atoms.get_potential_energy()
    print(f"Step {step:4d}, E = {E: .6f} eV")
```

实际工作中，更多会是：

- 用 DFT + ASE 采样少量高精度数据；
- 把数据喂给 ML 势或力场；
- 大规模 MD 用 ML 势 / GROMACS / LAMMPS 去跑。



评审:有社保, 55岁以下符合1个条件可直接评中高级工程师职称

 以正科技教育

[查看详情](#)

4.4 ASE × GROMACS: 接口与适用场景

ASE 也提供了一个 `ase.calculators.gromacs.Gromacs` 计算器，可以用 GROMACS 做经典力场的能量 / 力计算，甚至用于 QM/MM 的 MM 部分。

△ 但要注意：**官方明确说过这个接口非常慢 (VERY SLOW)**，不适合大规模、长时间的纯 MD 生产跑，更适合：

- 小体系测试；
- QM/MM 场景下把大环境交给 GROMACS，前景区交给 DFT；
- 在一个 Python 工作流里，偶尔丢给 GROMACS 算一些经典能量 / 力。

4.4.1 最简单的 ASE-GROMACS 示例（含注释）

```
from ase import Atoms
from ase.calculators.gromacs import Gromacs

# 1. 构建一个简单水分子（仅演示）
atoms = Atoms(
    "H2O",
    positions=[
        (0.000, 0.000, 0.000),
        (0.000, 0.000, 0.96),
        (0.80, 0.00, -0.24),
```

```

    ],
    cell=[2.0, 2.0, 2.0], # 简单立方盒子
    pbc=True,
)

# 2. 设置 GROMACS 计算器
calc = Gromacs(
    # 下面这些参数会因你的版本和接口实现略有不同,
    # 这里只给出一些可能用到的关键设置思路:

    # 告诉 ASE 使用哪个 GROMACS 命令 (一般系统默认就够)
    # gmx_path="gmx",

    # 可以指定 GROMACS 的 mdp 相关参数 (步数、温度等)
    # nsteps=0 # 只算一次能量 / 力

    # 也可以指定拓扑 / 力场相关文件
    # topol="topol.top",
    # mdp="md.mdp",
)

atoms.calc = calc

# 3. 让 GROMACS 后端算一次能量/力
energy = atoms.get_potential_energy()
forces = atoms.get_forces()

print("Energy from GROMACS:", energy, "eV (单位取决于接口实现)")
print("Forces:\n", forces)

```

实际工程里，更合理的做法通常是：

- 用 **ASE 构建复杂体系**（例如碳基底 + Nafion + H₂O + O₂）；
- 用 `ase.io.write` 写出 `.gro` / `.pdb` 等格式；
- 后续大规模 EM / NVT / NPT / 生产 MD，都交给原生 GROMACS（`gmx grompp` + `gmx mdrun`）；
- 轨迹分析可以交给 MDAnalysis / GROMACS 本身 / ASE 再配合 Python 脚本。

五、ASE × 机器学习 / 深度学习：几条可落地路径 🚗

5.1 DFT 数据 → 机器学习势能面 (ML Force Field)

1. 用 ASE + DFT 生成带有能量、力标签的训练数据；

2. 用 DeepMD / MACE / NequIP 等训练 ML 势;
3. 把 ML 势包装成 ASE Calculator, 再挂回 `atoms.calc` 上做大体系 / 长时间 MD。

核心逻辑类似于:

```
from ase.io import read

from mymlff import MLCalculator # 假设你有一个封装好的 ML 势计算器

atoms = read("config_for_ml.vasp")
atoms.calc = MLCalculator("model.pt")

E_ml = atoms.get_potential_energy()
F_ml = atoms.get_forces()
```

这样同一套 ASE 脚本可以灵活切换“DFT 模式”和“ML 模式”。

5.2 高通量 + 主动学习: 让模型自己“挑样本”

构建一个简单的主动学习循环:

```
while not converged:

    # 1. 用当前 ML 模型在大空间里快速预测
    #     并挑选不确定性最高 / 最有潜力的结构
    candidates = select_candidates_by_uncertainty(model, huge_pool)

    # 2. 对这些候选结构, 用 DFT + ASE 精算
    new_data = []
    for atoms in candidates:
        atoms.calc = HighLevelDFT(...)
        E = atoms.get_potential_energy()
        F = atoms.get_forces()
        new_data.append((atoms, E, F))

    # 3. 加入训练集, 更新 ML 模型
    dataset.extend(new_data)
    model = retrain_model(dataset)
```

这个框架可以用在:

- 吸附能 / 过渡态搜索;
- 催化剂组成 / 掺杂 / 缺陷空间探索;

- 固态电解质、储氢材料设计等。

ASE 在这里承担的是：**结构载体 + DFT 调用 + 输入输出统一管理**。

5.3 ASE + 简单特征 + scikit-learn / XGBoost

不一定一上来就用 GNN，先用 ASE + 经典 ML 也是很实用的路线：

```
from ase.io import iread
import numpy as np

X, y = [], []

for atoms in iread("dataset.traj"):
    feats = []
    # 例：体密度作为一个最简单特征
    feats.append(len(atoms) / atoms.get_volume())
    # TODO：可以继续加配位数分布、平均键长等结构描述符
    X.append(feats)
    y.append(atoms.info["target"]) # 例如吸附能 / 形成能

X = np.array(X)
y = np.array(y)
```

后续就是标准的 ML 流程：

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()
model.fit(X, y)
```

六、小结：把 ASE 当“中枢”，把工具链串成一个系统 🧩

如果用一句话概括 ASE：



ASE 是一个原子模拟世界的“中枢神经系统”：它用 `Atoms` 和 `Calculator` 把结构、DFT、力场、GROMACS、ML 势、数据库和可视化全都串起来。

在实际课题组 workflows 中，你可以这样组合：

- **建模 & 调度：** ASE (+ ASR 等工作流扩展)
- **量化 / 力场引擎：** VASP / GPAW / QE / GROMACS / LAMMPS / ML 势
- **命令行与可视化：** `ase info` / `ase convert` / `ase gui` + X11 / DISPLAY
- **轨迹与统计：** MDAnalysis、GROMACS 工具、ASE 自带分析
- **机器学习与自动设计：** scikit-learn / XGBoost / PyTorch / 各类 ML 势 + ASE 工作流

推荐阅读：

你的分子动力学，还卡在第一步？Packmol 让你告别“手动摆分子”的原始时代

百万原子体系模拟自由？从成功部署GROMACS开始

谁在用你的服务器？调度系统说了算

文献管理神器——Zotero，用过的都说好！

小白也能看懂的 Python 入门指南（一）



聚焦AI/机器学习/大模型与催化材料计算设计的深度碰撞，
探索“原子世界+数据智能”的前沿革命。
商业合作/投稿请联系：1181306835@qq.com

公众号二维码

运营者二维码



嘻歪歪

“赠人玫瑰，手留余香”

钟意作者

科研工具 · 目录

上一篇

文献管理神器——Zotero，用过的都说好！

下一篇

从零到上手：科研人和工程师眼中的
Windows 终端 (CMD)



百度网盘会员大促，参与预热最高加赠SVIP90天



[查看](#)

留言

写留言

